



ITI · AI AGENTS DEVELOPMENT USING PYTHON TRACK

AI Agents Development & Agentic Automation

Eng. Fatma Elraey

Lecture 1 — Agent Core & Function Calling (The ReAct Framework)

Field	Detail
Instructor	Fatma Elraey
Course	AI Agents Development (3-Day Course)
Session	Day 1 of 3 — Lecture, 3 Hours
Focus	Understanding the anatomy of an AI agent, the ReAct loop, tool engineering with Pydantic schemas, and the native Python tool-calling execution loop.

Welcome!

This course starts where basic LLM API integration ends. Instead of asking a model for one answer, you will build systems that can inspect a goal, choose an action, call real tools, read the result, and continue until the task is complete. The developer still owns tool boundaries, validation, execution, and safety.

Today's Outline

- 1. The Anatomy of an Agent
- 2. The ReAct Reason-and-Act Loop
- 3. Tool Engineering with Python + Pydantic
- 4. Executing the Native Gemini Tool-Calling Loop

01

The Anatomy of an Agent

A normal LLM call is usually single-turn: input goes in, text comes out. An agent adds a control loop around the model. The model can decide that it needs information or an action, select an available tool, inspect the observation returned by that tool, and then continue.

Anatomy of an AI Agent



Figure 1.1 — Four core pieces: perception, planning, memory, and tools.

- **Perception** — what the agent can currently see: the user request, relevant context, tool descriptions, and previous state.
- **Planning** — the model chooses the next useful action or decides that it can answer directly.
- **Memory** — information retained across steps or turns. Memory can be short-term conversation state or persistent application state.
- **Tools** — real capabilities exposed by your application: Python functions, databases, APIs, search, files, email, and more.

example — model vs agent

PLAIN MODEL “What is order 104?” → it can only answer from prompt/model knowledge.

AGENT “What is order 104?” → calls `get_order(104)` → reads the database result → answers with grounded data.

PRO TIP An agent is not a magically smarter model. It is a model connected to state and tools, controlled by an execution loop that your application owns.

✓ QUICK CHECK

Q1. What new capability turns a single LLM call into an agentic workflow?

Answer: The ability to choose and use tools across a loop, while carrying useful state between steps.

Q2. Who owns the actual execution environment?

Answer: Your application. The model proposes actions; your code validates and executes them.

02 The ReAct Reason-and-Act Loop

ReAct is a practical mental model for agent behavior: Reason about the goal, Act through a tool, Observe the result, then repeat if more work is needed. In production systems, the model's private reasoning does not need to be shown to the user; what matters to your program is the observable action request and the tool result.

The ReAct Agent Loop

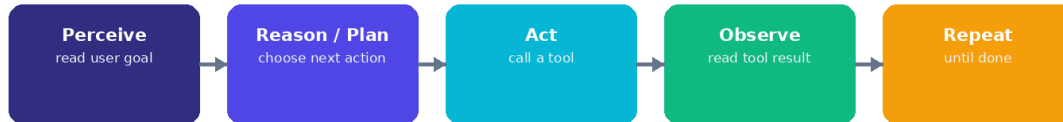


Figure 1.2 — ReAct repeats Action → Observation until the model can produce a final answer.

Instructor example — ReAct for an order request

USER "Check order 104 and tell me whether I should contact the customer."

ACTION 1 get_order(order_id=104)

OBSERVATION status=delayed, customer_id=22

ACTION 2 get_customer(customer_id=22)

OBSERVATION preferred_contact=email

FINAL Explain the delay and recommend the next customer-contact step.

The loop should always have clear stopping conditions. Common stop conditions are: the model returns a final answer without a tool call, a maximum number of steps is reached, a tool fails repeatedly, or the workflow pauses for human approval.

- **Action selection** — the model chooses from the tools you explicitly expose.
- **Observation** — your program converts a tool result into structured context for the next model step.
- **Iteration** — the next request includes the previous tool call and its result so the model can continue.
- **Stop** — do not allow unbounded loops; enforce a maximum step count and error policy.

✓ QUICK CHECK

Q1. What does the agent observe after an action?

Answer: The real output returned by the tool or external system.

Q2. Why should an agent loop have a maximum-step guard?

Answer: To prevent runaway cost, repeated failures, or endless tool-calling cycles.

03 Tool Engineering with Python + Pydantic

A good tool is a small, clear business capability with typed inputs and a predictable output. The model should not receive unrestricted access to arbitrary Python, SQL, or HTTP. Instead, expose narrow tools with explicit contracts.

Turning Code into a Safe Agent Tool



Figure 1.3 — A tool contract sits between the model and the real implementation.

```
# Typed tool input + narrow Python function
from pydantic import BaseModel, Field

class OrderLookup(BaseModel):
    order_id: int = Field(gt=0, description="Positive order identifier")

def get_order(order_id: int) -> dict:
    # Return a small, safe order summary by ID.
    # In a real app, this function may run a parameterized SQL query.
    fake_db = {104: {"status": "delayed", "total": 850}}
    return fake_db.get(order_id, {"status": "not_found"})
```

Pydantic is useful because the same schema can validate data before execution and document what the tool expects. That reduces invalid arguments and makes tool behavior easier to test.

- **Python function** — contains the real business logic.
- **Pydantic model** — validates tool arguments and can express required fields, ranges, enums, and descriptions.
- **SQL tool** — should use parameterized queries and expose only the operation the agent actually needs.
- **FastAPI endpoint** — can expose a tool behind a service boundary when the capability belongs to another application.

```
# example — bad tool vs good tool

TOO BROAD run_sql(query: str) — lets the model submit arbitrary SQL.
SAFER get_order(order_id: int) — exposes one read-only business capability.

TOO BROAD call_url(url, body) — arbitrary network access.
SAFER create_support_ticket(title, severity) — validated, auditable action.
```

PRO TIP The best agent tools are boring on purpose: small, deterministic, typed, auditable, and easy to test without the LLM.

✓ QUICK CHECK

Q1. Why is a narrow `get_order` tool safer than `run_sql`?

Answer: It limits the model to one approved capability instead of arbitrary database commands.

Q2. What does Pydantic add to tool engineering?

Answer: Typed validation and a clear schema for tool arguments.

04 Executing the Native Gemini Tool-Calling Loop

Gemini function calling follows the same four-part contract:

1. declare tools,
2. ask the model,
3. execute the selected function in Python,
4. then send the function result back so the model can continue.

The model does not directly execute your local Python function.

```
# Manual tool-calling cycle with Gemini
import os
from google import genai
from google.genai import types
from dotenv import load_dotenv

load_dotenv()
client = genai.Client(api_key=os.getenv("GEMINI_API_KEY"))

def get_order(order_id: int) -> dict:
    # Return the current status and total for an order.
    data = {104: {"status": "delayed", "total": 850}}
    return data.get(order_id, {"status": "not_found"})

config = types.GenerateContentConfig(
    tools=[get_order],
    automatic_function_calling=typesAutomaticFunctionCallingConfig(disable=True),
)

contents = [
    types.Content(role="user", parts=[types.Part(text="Check order 104")])
]

response = client.models.generate_content(
    model="gemini-3.5-flash-lite",
    contents=contents,
    config=config,
)

tool_call = response.candidates[0].content.parts[0].function_call
result = get_order(**tool_call.args)
```

```

contents.append(response.candidates[0].content)

# Fixed: Removed the 'id' keyword argument
contents.append(types.Content(
    role="user",
    parts=[types.Part.from_function_response(
        name=tool_call.name,
        response={"result": result}
    )],
))

final = client.models.generate_content(
    model="gemini-3.5-flash-lite",
    contents=contents,
    config=config,
)

print(final.text)

```

Output should look like this:

```
Order 104 has a status of **delayed** and a total of **$850**.
```

The manual version is ideal for learning because every boundary is visible. In simpler cases, the Gemini Python SDK can also execute supplied Python functions automatically. For agent engineering, understanding the manual loop is still important because production systems often need validation, logging, permissions, retries, approval checks, or custom routing between the model request and the tool execution.

Instructor example — what each part owns

MODEL chooses `get_order` and extracts `order_id=104`.

APPLICATION validates arguments, checks permissions, logs the call, and executes `get_order`.

TOOL returns structured real-world data.

MODEL AGAIN uses that observation to produce the final user-facing response.

PRO TIP *Never execute a model-selected action blindly. Validate the tool name and arguments, enforce permissions, catch errors, and log important actions before execution.*

✓ QUICK CHECK

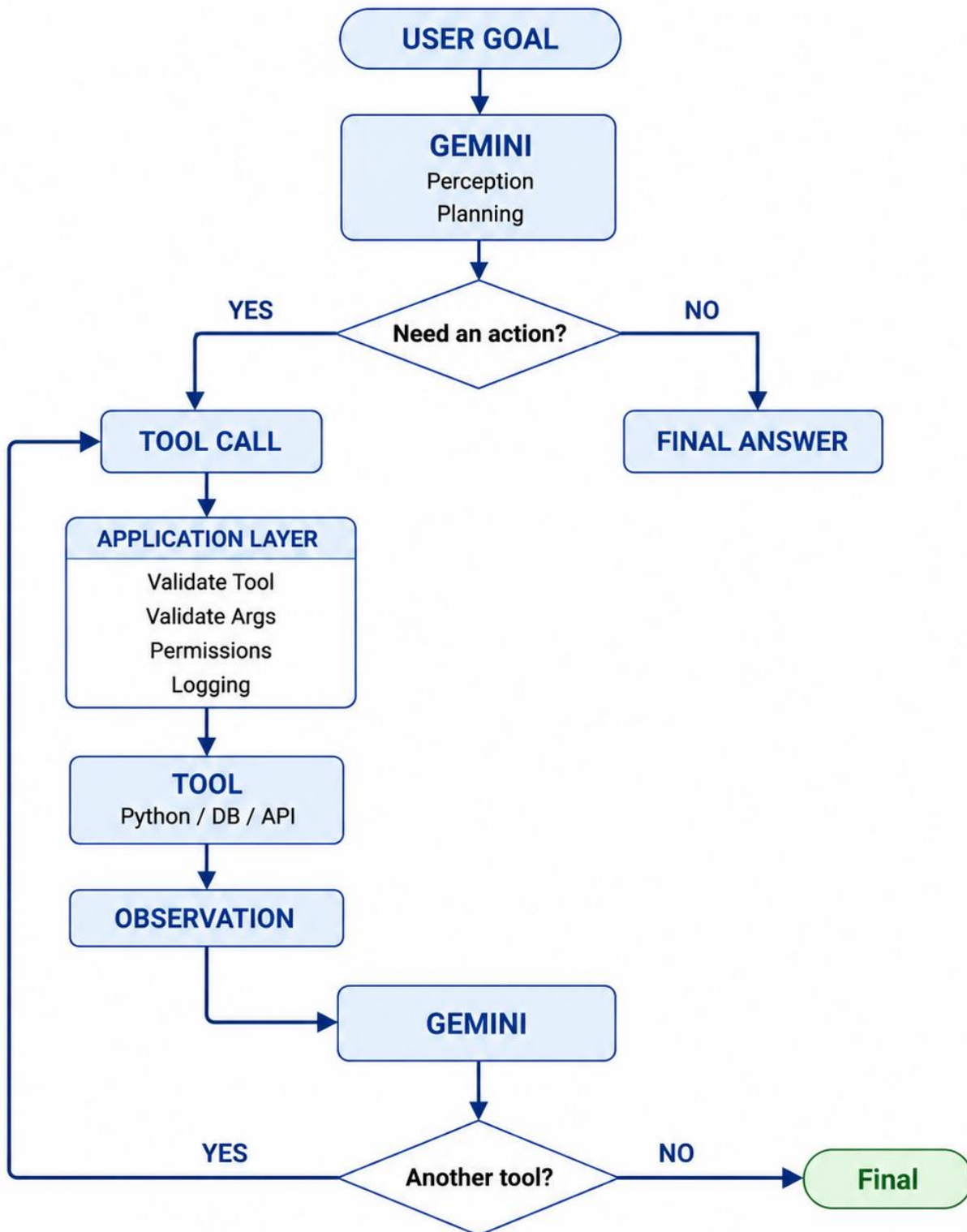
Q1. Does Gemini execute your local Python function in the manual loop?

Answer: No. Your application executes it and returns the result to the model.

Q2. Why send the tool result back to the model?

Answer: So the model can observe the real result and decide whether to answer or take another step.

Gemini Agent Execution Loop



99

Check Your Understanding

Work through these on your own before checking the marked answers.

Practice Question 1

Which combination best describes an agent?

- A. LLM only
- B. LLM + state + tools + an execution loop
- C. A database with a prompt
- D. A UI with no model

Practice Question 2

In ReAct, what normally follows an Action?

- A. A new training run
- B. An Observation from the tool
- C. A model restart
- D. A database migration

Practice Question 3

Which tool design is safest?

- A. `run_any_python(code)`
- B. `run_sql(query)`
- C. `get_order(order_id)`
- D. `call_any_url(url)`

Practice Question 4

What is Pydantic mainly doing in this course?

- A. Training the LLM
- B. Validating and documenting structured tool inputs
- C. Replacing Python functions
- D. Running n8n

Practice Question 5

Who actually executes a Gemini function call in the manual loop?

- A. The model
- B. Your Python application
- C. The browser
- D. The system prompt

Answers

Practice Question 1: B ✓ — Agent behavior comes from a model operating inside a controlled loop with tools and state.

Practice Question 2: B ✓ — The tool output becomes the observation used for the next decision.

Practice Question 3: C ✓ — A narrow typed business capability minimizes the action surface.

Practice Question 4: B ✓ — Pydantic models define and validate the inputs a tool accepts.

Practice Question 5: B ✓ — The model requests a tool; application code performs the real action.